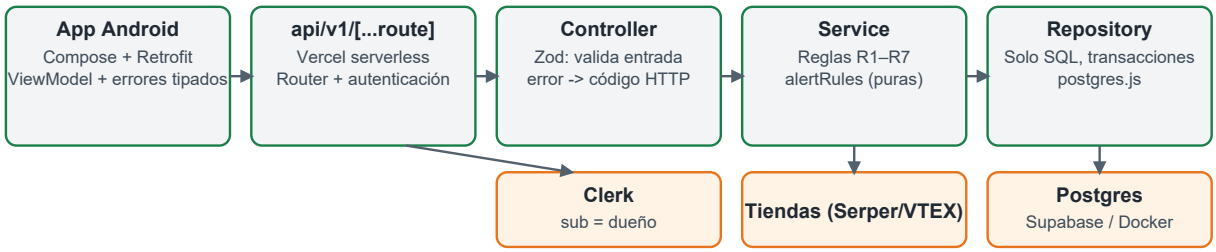
API REST de seguimiento de precios con alertas

Taller 3 · SI806 Desarrollo e integración de una API REST · Brian Curay, Marco Obispo, Niels Chaves · Repositorio: github.com/MarcOBL012/comparador-precios

1. Problema y alcance

Quien compra en tiendas peruanas no sabe cuándo baja el precio de un producto que quiere. La app *Comparador de Precios* ya identifica un producto con una foto y compara tiendas, pero esa consulta es puntual. Este taller agrega una **API REST propia** que permite **seguir un producto**, guardar la historia de sus precios en una base de datos relacional, y **generar alertas** cuando el precio alcanza el objetivo del usuario o llega a su mínimo histórico. Los precios provienen de consultas reales a las tiendas; no hay datos simulados. La API es nueva (versionada en `/api/v1`) y el cliente Android la consume desde una pestaña nueva, «Seguimiento».

2. Arquitectura



- **Capas separadas.** El *router* resuelve método y ruta; el *controller* valida la entrada con Zod (esquemas estrictos: un campo desconocido es error 400) y traduce errores de dominio a códigos HTTP; el *service* aplica las reglas de negocio; el *repository* es la única capa con SQL. Las decisiones de alertas viven en funciones puras (`alertRules.ts`), fáciles de probar sin base de datos.
- **Un solo punto de entrada.** Una función serverless `api/v1/[...route].ts` atiende toda la API con un router propio; así se evita el límite de funciones del plan gratuito de Vercel y las rutas con parámetros quedan en un solo sitio.
- **Identidad.** El dueño de cada recurso es el `sub` del token de Clerk, nunca un dato del cuerpo. Un recurso de otro usuario responde 404 (no 403) para no revelar que existe. Las tareas internas se protegen con `CRON_SECRET` comparado en tiempo constante.
- **Preparada para eventos.** Cada alerta escribe un evento en la tabla `domain_events` (patrón *outbox*) dentro de la misma transacción, lo que deja listo el enganche para notificaciones o procesos asíncronos sin acoplar el servicio.

3. Modelo de datos

Postgres (Supabase en producción, Docker en desarrollo), esquema versionado en `db/migrations/001–005`. 19 tablas y una vista, normalizadas a 3FN: ninguna columna depende de otra no clave. Por ejemplo, la alerta **no guarda** su mensaje (se deriva del tipo, la observación y el producto) ni el usuario (se deriva del seguimiento), y el estado del seguimiento es una clave foránea a un catálogo.

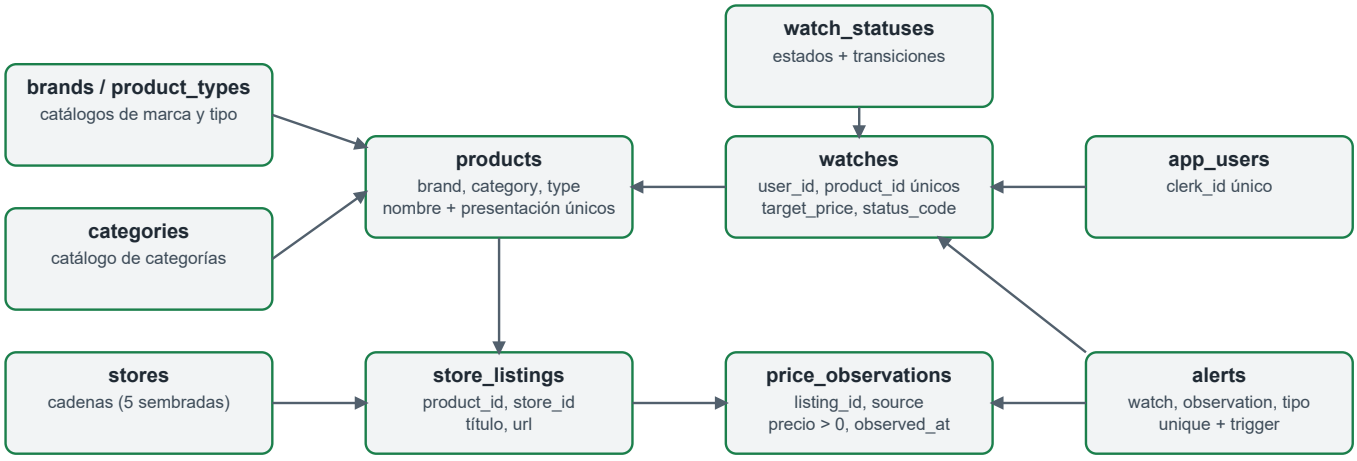


Tabla	Rol	Integridad
products, brands, product_types, categories	Catálogo de productos; el mismo producto no se duplica	`unique nulls not distinct (marca, nombre, presentación)`; FK a catálogos
stores, store_listings, price_observations	Dónde se vende y qué precio tuvo y cuándo (historial append-only)	precio > 0; borrado en cascada de listados y observaciones
app_users, watches	Un seguimiento por usuario y producto	`unique (user_id, product_id)`; objetivo > 0; FK a `watch_statuses`
watch_statuses, watch_status_transitions	Máquina de estados como datos	Trigger rechaza una transición no registrada, aunque se escriba por fuera de la API

Tabla	Rol	Integridad
alerts, alert_types	Alertas derivadas de una observación	`unique (observación, seguimiento, tipo)`: no se duplican; trigger exige que la observación sea del mismo producto
devices, push_deliveries, domain_events	Dispositivos, entregas de notificación y outbox	Preparadas para notificaciones push y procesamiento de eventos
identification_signatures, signature_confirmations	Confirmaciones de la comunidad (vista de consenso)	Un voto por usuario y firma; verificado con ≥ 2 votos y ≥ 66 %

4. Endpoints

Método y ruta	Qué hace	Respuestas
`POST /watches`	Sigue un producto (objetivo opcional)	201 + Location; 400, 401, 409, 422
`GET /watches`	Lista del usuario (filtro estado, límite, desplazamiento)	200; 400, 401
`GET /watches/{id}`	Detalle con mejor precio actual	200; 400, 401, 404
`PATCH /watches/{id}`	Cambia objetivo o estado (pausar, reactivar)	200; 400, 401, 404, 409, 422
`DELETE /watches/{id}`	Deja de seguir y borra sus alertas	204; 401, 404
`POST /watches/{id}/check`	**Operación de negocio:** consulta tiendas, guarda observaciones y decide alertas	200; 401, 404, 409, 429 + Retry-After, 502
`GET /watches/{id}/history`	Historial de precios y estadística (mínimo, máximo, mediana, tendencia)	200; 400, 401, 404
`GET /alerts`	Alertas del usuario, con filtro de no leídas	200; 400, 401
`PATCH /alerts/{id}`	Marca una alerta como leída o no leída	200; 400, 401, 404
`GET|POST /internal/check-due`	Cron diario: revisa los seguimientos que lo necesitan	200; 401

Todos los errores devuelven JSON `{error, codigo, detalles?}`; en 400 y 422 *detalles* indica el campo y el motivo. Cuerpos de ejemplo, esquemas y códigos de cada ruta están en `docs/api/openapi.yaml`, y una colección de Postman con 26 peticiones (éxitos y fallos) en `docs/api/postman_collection.json`.

5. Reglas de negocio

Regla	Descripción	Resultado
R1	Un seguimiento por usuario y producto	409 `ya_seguido`
R2	Máximo 20 seguimientos activos por usuario	409 `limite_seguimientos`
R3	Una revisión cada 15 minutos por seguimiento (protege a las tiendas y la cuota)	429 con `Retry-After`
R4	Si el mejor precio ≤ objetivo se crea la alerta *objetivo_alcanzado* y el seguimiento pasa a *alcanzada*	Alerta + cambio de estado, en una transacción
R5	Alerta *minimo_historico* si el precio supera el menor de al menos 3 observaciones previas	Alerta
R6	Se descartan precios atípicos: menos del 40 % de la mediana histórica, con al menos 3 observaciones previas (listados mal emparejados)	No se guarda la observación
R7	Transiciones: activa ↔ pausada; alcanzada → activa | pausada; solo el sistema pasa a alcanzada; reactivar una alcanzada exige un objetivo menor	409 / 422

La revisión (`recordCheck`) usa una transacción con bloqueo de fila (`for update`): observaciones, alertas, cambio de estado y evento se guardan juntos o no se guarda nada. Así, dos revisiones simultáneas no duplican alertas.

6. Validación y manejo de errores

- **Servidor:** Zod valida cuerpo, parámetros de ruta y consulta (UUID, rangos, tipos); el controller devuelve 400 con detalle por campo. Los errores de dominio se mapean en un solo sitio (`lib/http/respond.ts`): 401, 404, 405, 409, 422, 429, 502 (tiendas no responden), 503 (sin base de datos) y 500 sin filtrar detalles internos.
- **Cliente Android:** `WatchRepository` convierte cualquier fallo en un `ApiError` tipado: *Network* (sin conexión o tiempo agotado), *BadResponse* (JSON con forma inesperada), *Server* (5xx), más *Unauthorized*, *NotFound*, *Conflict*, *Validation* y *RateLimited*. La pantalla muestra un mensaje distinto para cada caso y ofrece reintentar.

7. Cliente Android

Consume 9 de los endpoints desde Retrofit con kotlinx.serialization. Pantalla *Seguimiento*: lista con estado, mejor precio y objetivo; acciones revisar ahora, pausar o reactivar, editar objetivo, ver historial y eliminar; alertas con contador de no leídas. Tras una comparación, la tarjeta «Seguir este precio» crea el seguimiento. El *ViewModel* expone un `StateFlow` y libera los botones ante cualquier error, incluido el 429.

8. Pruebas

Nivel	Cantidad	Qué cubre
Reglas puras (`alertRules`)	15	Objetivo alcanzado, mínimo histórico, outliers, historial insuficiente, mensajes
Servicio con repositorio en memoria	15	R1–R3 y R7, propiedad del recurso, estados inválidos
API completa (router + controller + servicio) y autenticación	17	Códigos 200/201/204/400/401/404/405/409/422/429, JSON de error, validación, modo desarrollo ignorado en Vercel
Integración contra Postgres real	13	CRUD persistido, producto compartido entre usuarios (normalización), R3, R4 y R6, atomicidad de la revisión, cascadas, cron, 404 en recursos ajenos
Android (repositorio y ViewModel)	23	Mapeo de 401/404/409/400/422/429/502/500, sin red, JSON inválido; estados de la pantalla

Backend: `npm test` (Vitest; las de integración corren con `TEST_DATABASE_URL`) y `npm run build` (tsc). Android: `./gradlew :app:testDebugUnitTest`. Todas pasan al momento de la entrega.

9. Cómo ejecutarlo

El README del repositorio detalla los pasos: `npm run db:up` levanta Postgres con Docker, `npm run db:migrate` aplica el esquema, y la API local se prueba con la colección de Postman. En producción se requieren dos variables de entorno en Vercel (`DATABASE_URL`, del *Transaction pooler* de Supabase, y `CRON_SECRET`); `vercel.json` programa la revisión diaria.